

CSA5801

DEVSECOPS ENGINEERING AND APPLICATION SECURITY

CO4 AT

Abinesh M 192465050

CSA5801 - DevSecOps Engineering and Application Security

IAM Relationship Modelling & GitHub Integration

Practical Overview

IAM Relationship Modelling and GitHub Integration Using RStudio

IAM Modelling

Represent users, roles, permissions and GitHub resources as relationships.

RBAC Simulation

Test whether specific roles are authorized to perform GitHub-related actions.

GitHub Integration

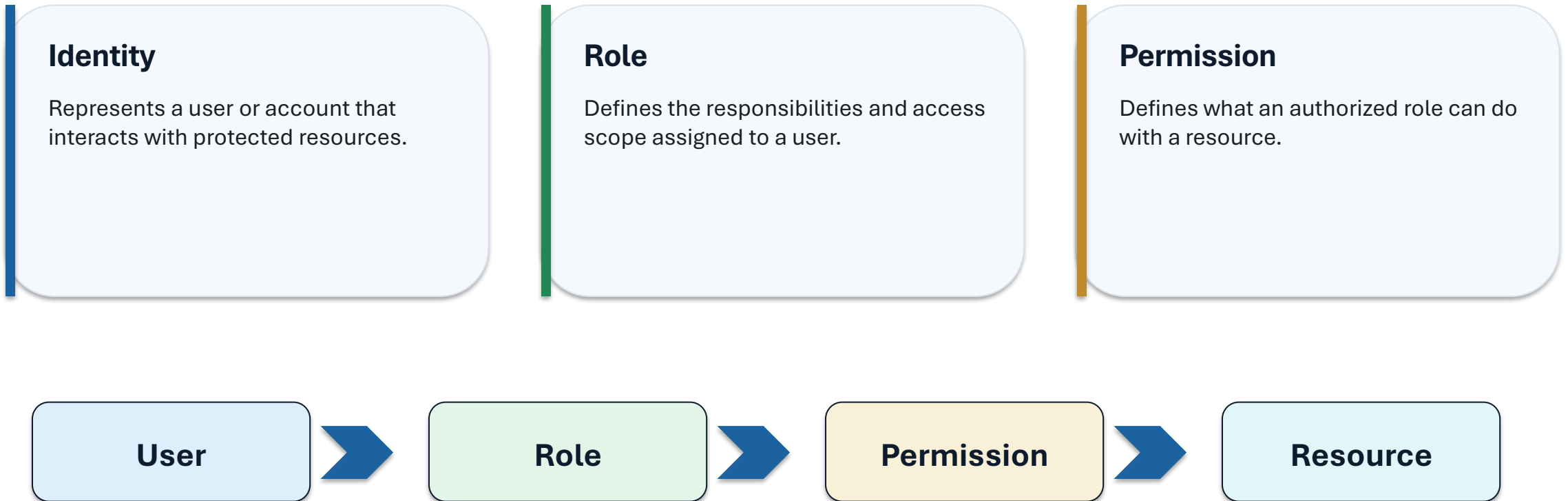
Use Git from RStudio to commit and push the practical project to GitHub.

Software: RStudio, R, Git and GitHub

Visualization: DiagrammeR / grViz()

Core flow: User → Role → Permission → Access Decision → GitHub Resource

Identity and Access Management (IAM)



IAM supports controlled access, least privilege and accountability in secure software development.

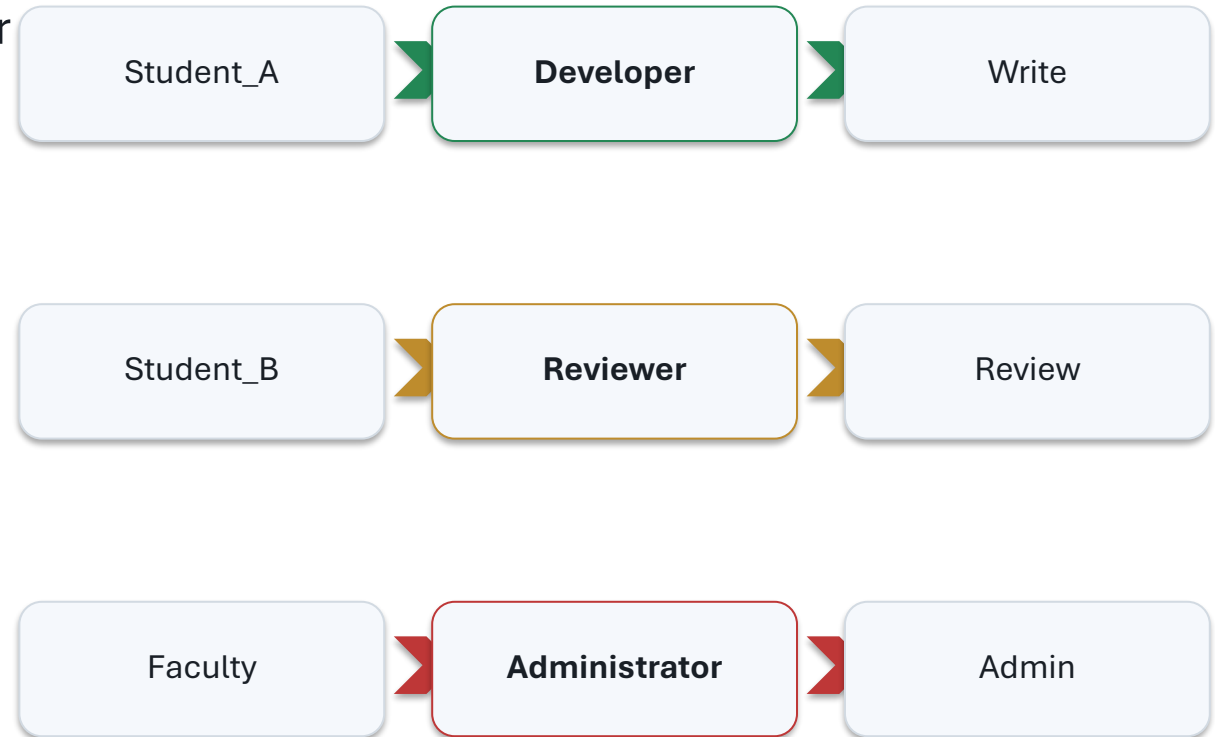
Role-Based Access Control (RBAC)

RBAC assigns permissions through roles rather than giving every user direct access.

Developer: code development and approved repository actions.

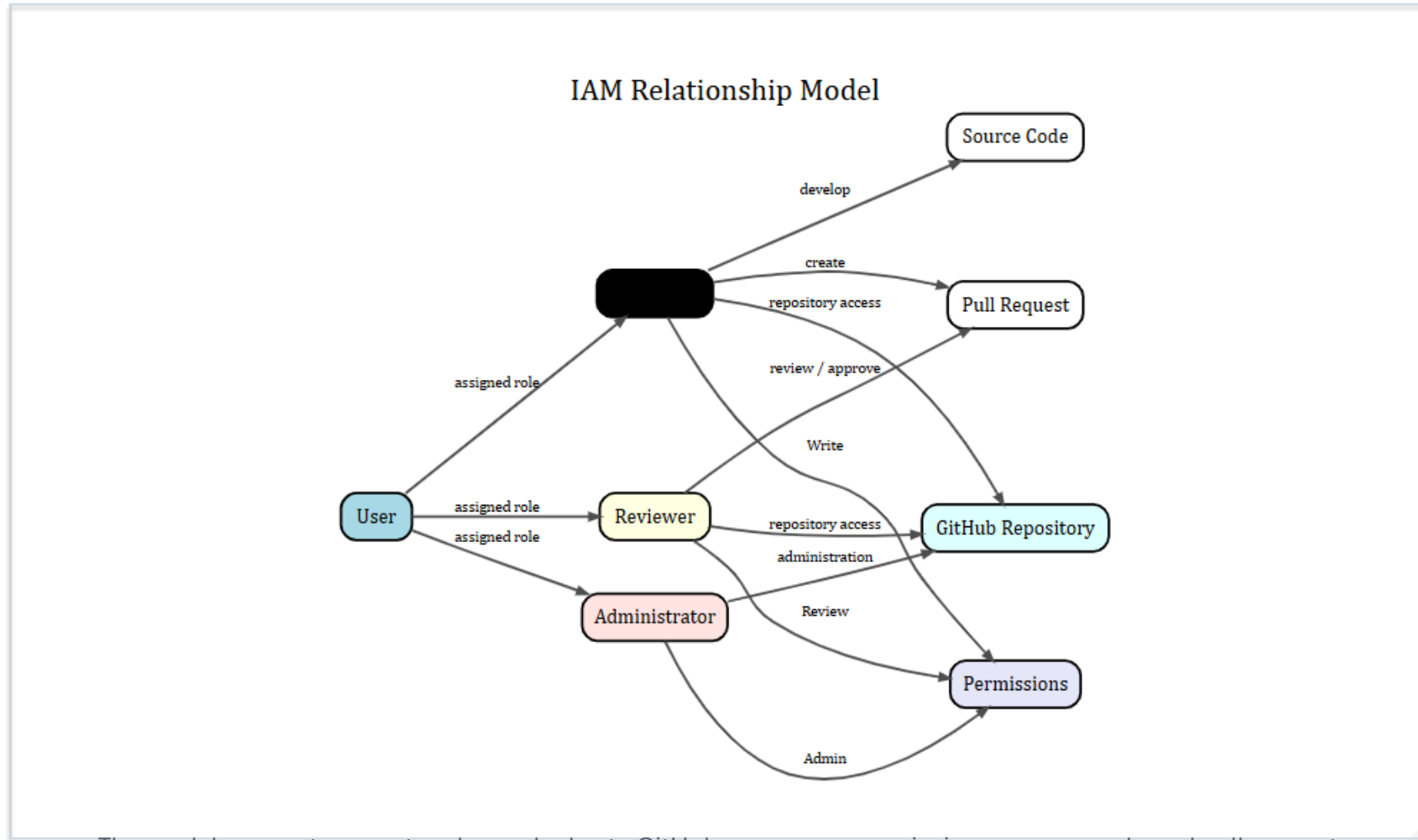
Reviewer: pull-request review and approval.

Administrator: repository administration and broad control.



IAM Relationship Model

DiagrammeR implementation output



The model connects users to roles and roles to GitHub resources, permissions, source code and pull requests.

Implementation Using RStudio and DiagrammeR

RStudio

Development environment used to write, run and manage the R practical.

DiagrammeR

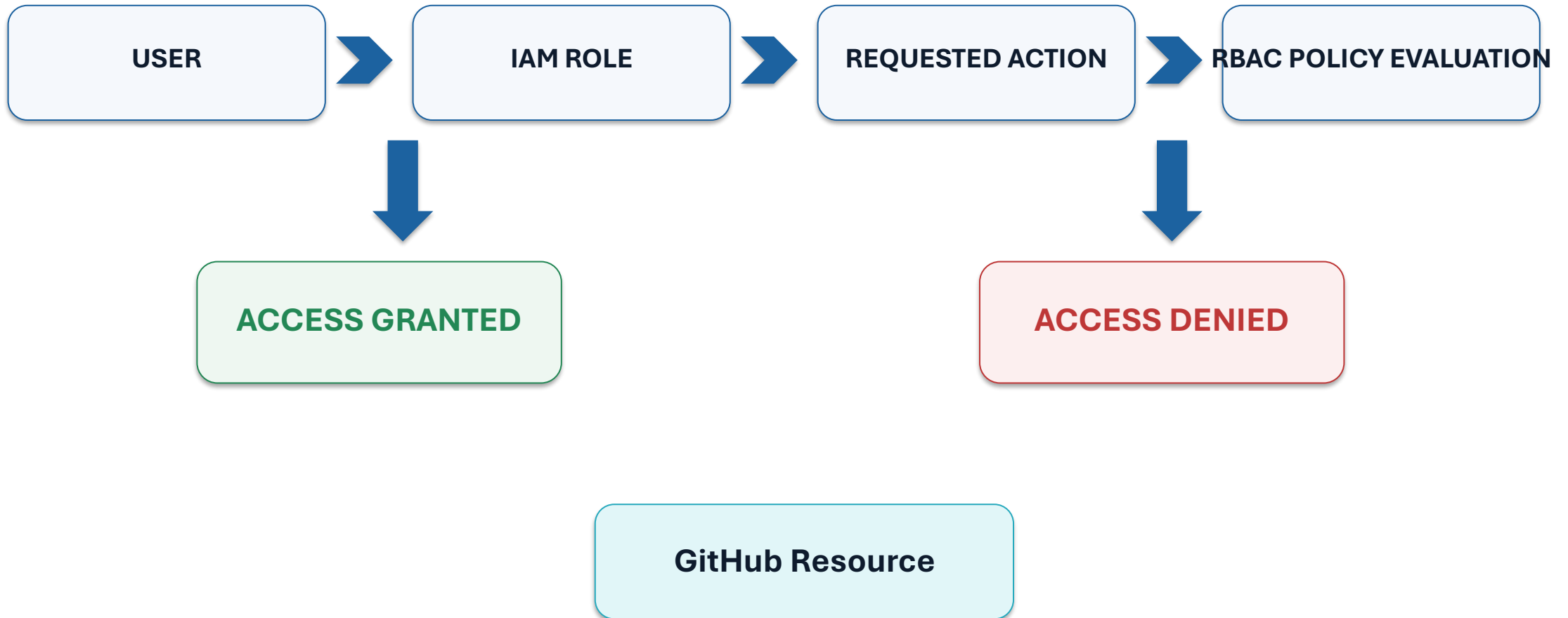
R package used to construct the IAM relationship graph.

grViz()

Generates the graph from a DOT-style relationship definition.

```
library(DiagrammeR)
grViz("digraph IAM { ... }")
User → Role → GitHub Resource
```

IAM Access-Control Workflow



Granted requests proceed to the permitted GitHub resource; denied requests are blocked by the simulated RBAC policy.

IAM User, Role and Permission Mapping

Actual R console output

User	Role	Permission
Student_A	Developer	Write
Student_B	Reviewer	Review
Faculty	Administrator	Admin

Interpretation

Each user is mapped to one role, and the role determines the permission scope used by the access-control simulation.

Role-Based Permission Matrix

Developer

Commit • Push • Create Branch

Reviewer

Review Pull Request • Approve Pull Request

Administrator

Commit • Push • Create Branch • Review Pull Request • Approve Pull Request • Delete Repository • Manage Repos

RBAC Access-Decision Simulation

Five required practical test cases

Test	Role	Action	Decision
T01	Developer	Commit	ACCESS GRANTED
T02	Developer	Delete Repository	ACCESS DENIED
T03	Reviewer	Approve Pull Request	ACCESS GRANTED
T04	Reviewer	Push	ACCESS DENIED
T05	Administrator	Delete Repository	ACCESS GRANTED

Access-Control Results and Analysis

05

Total Requests

03

Granted

02

Denied

Developer can perform development-oriented actions such as Commit.

Developer is denied repository deletion, demonstrating restricted privilege.

Reviewer can review/approve pull requests but cannot Push.

Administrator is granted the broadest control in the simulation.

The results demonstrate role-based authorization rather than unrestricted access.

Git and GitHub Integration



Git is enabled through RStudio's Git/SVN settings.

iam_github.R is committed with the message: “IAM relationship modelling completed”.

The project is pushed to the GitHub repository and verified online.

Submission Evidence and Conclusion

Evidence Captured

- ✓ IAM relationship diagram
- ✓ IAM user-role-permission table
- ✓ RBAC access-control output
- ✓ Git commit evidence
- ✓ GitHub repository evidence
- ✓ Successful push evidence

Conclusion

The practical demonstrates how IAM relationships and RBAC rules can be modelled in R, tested through access decisions, visualized using DiagrammeR, and integrated with Git and GitHub as part of a DevSecOps workflow.

IAM → RBAC → Access Decision → Git → GitHub